

# 題目：即時預警與決策自動化：Docker/Zabbix 網路系統監測應用實戰

級別：112 班級：資工三甲、乙

指導老師：林易泉 教授

組長：林采儀 學號：41243103

組員：伍翊瑄 學號：41243101

組員：許舜曄 學號：41243231

## 摘要

本專題旨在解決網路維護中常見的「被動式維護」核心痛點，並實作一套具備主動預警與自我修復能力的 **Zabbix NMS** 模擬監測系統。

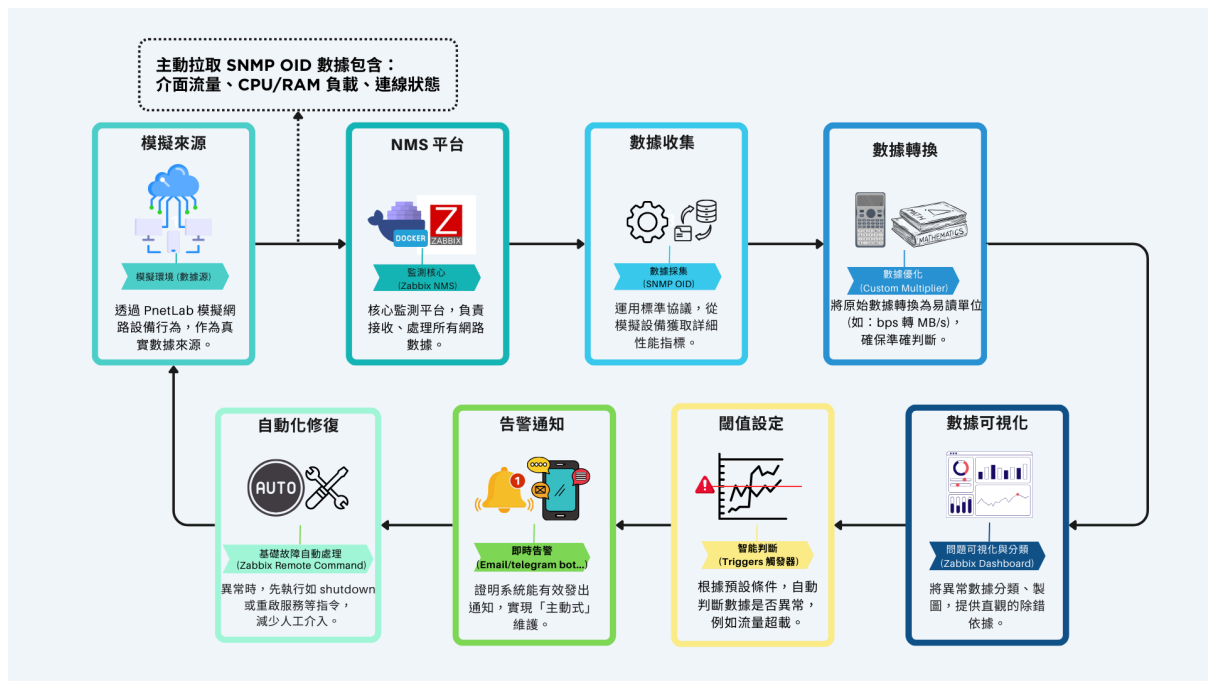
技術方案上，專案部署於 **PNETLAB** 虛擬化平台，採用 **Docker** 容器化部署 Zabbix 核心服務，確保系統的高移植性。我們使用 **SNMP** 模擬器取代昂貴的實體 Switch 設備，實現低成本、高擬真的監測環境。

核心貢獻聚焦於兩大技術突破：

1. 數據標準化：透過 Zabbix Pre-processing 機制，精準地將 SNMP OID 採集的原始網路流量數據轉換為實務所需的 **Mbps** 單位。
2. 自動化閉迴路：系統實作了基於 Zabbix Actions 的 **Remote Command** 自動化響應機制，演示了在網路指標異常時，系統能夠主動執行應急修復腳本，完成從「故障偵測」到「自動響應」的完整事件管理閉迴路。

本專題成功建立了一個可重現、可驗證的網路監測實戰平台，證明了從傳統被動通知轉向預測性監控與自動化應急的可行性，為網路設備監測領域提供了具備高度實務價值的解決方案。





## 研究動機與目的

### 研究動機：從被動維護到主動預警

想像一下，在一個擁有數百名學生的宿舍樓中，當核心公共設備，例如洗衣機或烘乾機發生故障時，造成的影響是巨大的：學生需要花費大量時間尋找可用設備，生活品質受到影響，而管理員往往是接到大量投訴後才被動進行維修。這種「事後補救、被動式維護」的模式，導致了嚴重的時間成本、生活影響與管理壓力。

這種「被動式維護」的效率黑洞在任何基礎設施中都存在。本專題借鑑「洗衣機故障」所暴露出的延遲與被動的維護模式，並將其嚴重性類比至核心網路設備。在全球化與數位轉型的趨勢下，網路設備（如核心交換器 **Switch**）的穩定性是業務連續性的關鍵。一旦核心 Switch 發生故障或性能下降，造成的營運損失遠超生活中的不便。

傳統依賴人力巡檢的網路維護方式，往往反應遲緩，難在故障發生前即時預警或快速定位問題。

因此，本專題的動機便是要解決這一「被動式維護」的共同痛點：我們將以網路核心設備為實作對象，旨在展示從被動維護轉變為主動式、預警式監控的必要性與一套完整的技術實作方法。

## NMS 競品與市場定位分析

本章旨在分析當前網路管理系統 (NMS) 市場趨勢與主流產品的技術定位，藉此確立本專題方案 (Zabbix/PNETLAB) 的技術價值與選擇的合理性。

### 1. NMS 競品市場概況

目前市場上的 NMS 產品主要分為開源 (如 Zabbix、Nagios) 和商業解決方案 (如 SolarWinds、Datadog)。值得注意的是，商業解決方案已經從傳統的「監測」邁向「可觀測性 (Observability)」和「AIOps」等下一代功能。

| 競品名稱                   | 市場定位       | 核心優勢                                               |
|------------------------|------------|----------------------------------------------------|
| Zabbix (本專題)           | 開源、高度客製化   | 免費、高度可擴展、強大的閾值告警機制。                                |
| SolarWinds             | 傳統 NMS 領導者 | 強大的網路性能監測 (NPM)、自動發現與拓模圖繪製、豐富的設備模板。                |
| ManageEngine OpManager | 多功能整合方案    | 支援超過 11,000 個設備模板，易於使用，涵蓋網路、伺服器、應用程式。              |
| Datadog / Dynatrace    | 雲原生、全棧可觀測性 | 整合 Metrics、Logs、Traces，具備強大的 AI 引擎進行自動化根因分析 (RCA)。 |

### 2. 商業競品達到的「下一代」功能

商業競品在三個關鍵領域已經達到了超越傳統 NMS 的程度，這些功能是本專題未來發展的指引方向：

#### 2.1 從「閾值告警」到「AIOps 預測分析」

本專題的解決方案基於靜態閾值。而領先的商業 NMS 則利用機器學習 (ML) 實現了 AIOps (Artificial Intelligence for IT Operations)：

- **異常偵測 (Anomaly Detection)**：系統能夠學習設備的正常行為基準線，自動識別與基準線的細微偏差，實現預測性維護。
- **事件關聯與降噪**：AI 將數百個獨立警報聚合成單一的事件，並隔離出根本原因 (Root Cause)，大幅降低維護人員的「告警疲勞 (Alert Fatigue)」。

#### 2.2 全棧可觀測性 (Full-Stack Observability)

現代商業平台將監控深度擴展至整個應用棧：

- 整合 **Metrics, Logs, Traces**: 在單一平台中統一收集和分析性能指標、系統日誌 Logs 和應用程式追蹤數據 Traces。
- 動態拓撲與相依性映射: 自動繪製整個網路和應用程式服務的相依性地圖, 即時顯示故障對業務的影響範圍。

## 2.3 自動化修復與服務集成

商業 NMS 不僅是通知, 還能自動採取行動。這也是本專題規劃實作 **Zabbix Actions** 的核心方向:

- 自動化工作流 (**Automated Workflows**): 故障發生時, 系統可自動運行 Script 嘗試自我修復 (例如重啟服務)。本專題即以 **Zabbix Remote Command** 實作此概念。
- **ITSM/Service Desk** 集成: 自動在服務管理系統 (如 Jira) 中開立工單 Ticket, 將監控與維護流程無縫連接, 完成從警報到維護追蹤的完整閉迴路。

## 3. 本專題的技術定位與貢獻

綜合考量市場趨勢與成本效益, 本專題確立了 Zabbix/PNETLAB 的組合, 旨在以最小的成本, 穩固掌握所有商業 NMS 的核心技術基礎 (採集、標準化、響應), 並為未來發展奠定基石:

- 解決基礎連線問題: 透過 Docker 固定 IP 和 PNETLAB, 解決了數據採集層的穩定性問題。
- 實現數據標準化: 利用 Custom Multiplier 實現了數據標準化層, 證明了將原始 OID 數據轉化為業務指標 (如 Mbits/sec) 的能力。
- 基礎告警閉迴路: 成功演示了 Trigger 判斷和 Action 通知機制, 完成了監控與響應的閉迴路。

## 核心技術選用分析 (Why Zabbix)

面對從被動維護轉變為主動式、預警式監控的必要性, 同時考量到實務上難以取得或大規模部署多種實體設備進行 NMS 測試與教學驗證, 本專題透過精心選擇技術組合來克服限制:

- **Zabbix** 平台: 選擇 Zabbix 而非 Nagios 等工具, 是因為 Zabbix 提供一站式的數據採集、圖形化、告警觸發 (Triggers) 和通知 (Actions) 功能, 在歷史數據趨勢分析和效能監測上更具優勢, 符合專案對數據深度的要求。
- **Docker** 容器化: 選擇 Docker 實現 **Zabbix Server** 的部署, 是為了確保環境配置的高效率、一致性與可移植性, 讓整個 NMS 平台可以輕鬆地在不同環境中快速重現。
- **PNETLAB** 與 **SNMP** 模擬器: 選擇在 PNETLAB 建立虛擬網路拓撲並搭配 SNMP 模擬器, 是為了在低成本環境下實現高度擬真性。這也是為了直接驗證網路

Switch 設備監控的核心技術 **SNMP OID** 採集和 **Custom Multiplier** 轉換, 這是使用 Agent 監測方式無法達成的目標。

. **EWMA** 異常偵測模型: 在資料回饋層加入 EWMA (Exponentially Weighted Moving Average) 統計模型, 利用 Zabbix 所提供的歷史資料與即時監控值, 進行平滑化計算並偵測偏離趨勢的異常行為。EWMA 能有效強化 Zabbix 既有告警機制, 使監控系統具備更敏捷的變化感知能力, 達成「由被動反應轉向主動預警」的核心目標。

## 研究目的

本專題的核心目的在於 實作並驗證 一套基於 **Docker/Zabbix** 架構的 NMS 監測解決方案, 具體目標如下:

1. 實作 **PNETLAB** 虛擬化環境: 在 **PNETLAB** 平台 上部署基礎網路拓撲, 並架設基於 Docker 容器化技術的 **Zabbix NMS** 平台, 確保系統具備高可移植性與易維護性。
2. 建立網路設備模擬環境: 運用 **SNMP** 模擬器 部署虛擬 Switch 設備節點, 並能夠餵入預設數據 模擬真實網路流量與效能數據, 取代實體設備進行測試。
3. 精確實作 **SNMP** 數據收集: 成功配置 Zabbix Agentless 監測, 掌握並應用 **SNMP OID** 技術, 實現對 Switch 關鍵指標 (如流量、CPU/Memory 使用率) 的精準數據採集, 並正確處理如網路流量的 單位換算 問題。
4. 建立自動化告警與異常偵測機制: 設定 **Triggers** 條件, 並透過修改模擬器數據, 主動演示網路故障 (如高流量、設備異常) 的發生, 並成功觸發 Zabbix 的告警通知。同時, 前端採用 EWMA 統計模型自動偵測突發流量與設備異常, 達成 主動監測與預警 的核心目標。
5. 建立自動化響應機制與閉迴路: 配置完整的 **Actions** (動作), 不僅發送通知, 更規劃實作輕量級的自動化修復腳本 (例如: 執行 Zabbix Remote Command 嘗試重啟模擬服務或執行基礎網路指令), 演示從「告警發生」到「自動化響應」的完整監測生命週期, 達成減少人工介入的核心目標, 並體現與 **ITSM/工單系統** 整合的可能性。
6. 建立前端 **Dashboard** 顯示與操作介面: 利用 **React + Chart.js** 提供即時監控視覺化, 包括 Switch 流量、CPU/Memory 使用率及告警事件。透過 **WebSocket** 即時更新, 讓使用者能快速掌握設備狀態、歷史趨勢與異常事件, 提升操作效率與決策支援。

## 研究方法與進行步驟

---

本專題是在 **PNETLAB** 虛擬化平台上建立網路拓撲。這能確保 NMS 平台 (Zabbix) 與模擬的網路設備 (Switch/SNMP Simulator) 在一個高度擬真的網路環境中進行通訊與監測。

## 第一階段: NMS 平台與 PNETLAB 環境建構 (Infrastructure Setup)

| 步驟                  | 說明                                                                       |
|---------------------|--------------------------------------------------------------------------|
| 1.1 拓撲設計與 IP 規劃     | 在 PNETLAB 上規劃網路拓撲, 包含 Zabbix Server (Docker) 節點、SNMP 模擬器節點。配置各節點的 IP 位址。 |
| 1.2 平台部署            | 導入並啟動 Zabbix Server (Docker) 節點, 配置運行環境。                                 |
| 1.3 模擬器配置           | 部署 SNMP 模擬器, 設定模擬器的 IP 與 SNMP Community String, 並預載 Switch 相關 OID 數據。    |
| 1.4 PNETLAB 內部連通性驗證 | 從 Zabbix Server 節點向模擬器進行 Ping 測試 及 SNMP Walk/Get 測試。                     |

## 階段二: 數據收集與客製化處理

| 步驟              | 說明                                                                                           |
|-----------------|----------------------------------------------------------------------------------------------|
| 2.1 OID 選定與樣板配置 | 選定 Switch 的關鍵監測 OID, 並在 Zabbix 中建立 Template (樣板) 與 Items (監測項), 設定數據採集頻率。                    |
| 2.2 流量單位轉換實作    | 針對網路流量數據, 在 Item 的 Pre-processing (預處理) 步驟中實作 Custom Multiplier 邏輯, 將 Byte 轉換為 Mbps 等實務常用單位。 |
| 2.3 數據穩定性驗證     | 將樣板連結到模擬器 (Host), 確認 Zabbix 可持續、穩定且正確地收集並繪製數據圖表。                                             |

## 階段三: 告警機制與故障模擬驗證

| 步驟                | 說明                                                                       |
|-------------------|--------------------------------------------------------------------------|
| 3.1 Triggers 告警設定 | 針對流量超載、設備 CPU 過高等情境，設定 Triggers 條件與嚴重等級，遵循業界閾值原則。                        |
| 3.2 告警動作配置        | 配置 Actions (動作) 的 <b>Media Type</b> ，例如設定 Email 或其他通知方式，確保基礎告警能即時發送。     |
| 3.3 故障情境演示        | 手動修改 SNMP 模擬器中的數據，使監測數值超過設定的 Triggers 門檻。                                |
| 3.4 基礎流程驗證        | 驗證 Zabbix 發出告警通知，隨後將模擬數據恢復正常，確認 Zabbix 轉為 Problem Resolved 狀態，完成基礎監測閉迴路。 |

#### 階段四：自動化響應與流程閉迴路實作 (Self-Healing Layer)

這是新增的重點環節，用於實作和驗證您的進階目標。

| 步驟                           | 說明                                                                                 |
|------------------------------|------------------------------------------------------------------------------------|
| 4.1 自動化修復腳本開發                | 針對高嚴重性告警(例如:CPU 超載 95%)，設計並實作輕量級的自動化修復腳本，作為初階應急手段。                                 |
| 4.2 Zabbix Remote Command 配置 | 在 Zabbix Actions 中，配置 <b>Remote Command</b> (遠端命令)，使系統能在特定高嚴重性告警觸發時，自動對模擬設備執行修復腳本。 |
| 4.3 自動化閉迴路驗證                 | 重新演示故障情境，驗證系統是否同時成功發出告警並執行遠端命令。並監測數據，確認自動化修復嘗試後指標是否回歸正常，完成主動響應與自我修復閉迴路的驗證。         |
| 4.4 ITSM/維護流程規劃              | 文件上說明或架構圖上標註如何將此告警資訊格式化，以便未來對接 <b>ITSM/工單系統</b> ，展示與企業維護流程的整合性。                    |

#### 階段五：前端資料視覺化與即時呈現

| 步驟 | 說明 |
|----|----|
|----|----|

|                      |                                                                                      |
|----------------------|--------------------------------------------------------------------------------------|
| 5.1 即時資料更新           | 透過 <b>WebSocket</b> 與 <b>Node.js</b> 中介層連線, 即時接收後端監控資料, 確保 Dashboard 即時更新, 顯示最新監控狀態。 |
| 5.2 Global Alerts 面板 | 顯示當前告警事件與狀態 (Critical / Warning), 使用顏色或圖示突出異常, 讓使用者快速掌握重要訊息。                         |
| 5.3 歷史趨勢查詢           | 提供 Logs 或 History 頁面, 讓使用者能查詢過去一段時間的設備數據與趨勢, 支援容量規劃與異常事件分析。                          |

## 階段六: 成果整理與專題報告撰寫 (Finalization)

| 步驟             | 說明                                         |
|----------------|--------------------------------------------|
| 6.1 數據趨勢分析與文件化 | 整理、分析並擷取各階段的監測數據圖表, 將專案的實作過程和關鍵技術細節形成完整文件。 |
| 6.2 專題報告與簡報製作  | 完成專題計畫書、最終成果報告與演示簡報的撰寫與製作。                 |

## 執行進度(甘特圖)

---



| 專題階段          | 任務名稱                       | 10月 | 11月 | 12月 | 1月 | 2月 | 3月 | 4月 |
|---------------|----------------------------|-----|-----|-----|----|----|----|----|
| I. 環境建構       | 文獻、OID 搜集、PNETLAB規劃        |     |     |     |    |    |    |    |
|               | Zabbix/Docker 平台部署         |     |     |     |    |    |    |    |
|               | SNMP 模擬器部署與連通性測試           |     |     |     |    |    |    |    |
| II. 數據收集      | Zabbix 樣板與 Items 基礎配置      |     |     |     |    |    |    |    |
|               | Custom Multiplier 實作與最佳化   |     |     |     |    |    |    |    |
|               | 數據穩定性與圖表驗證                 |     |     |     |    |    |    |    |
| III. 告警機制     | Triggers 告警條件設定            |     |     |     |    |    |    |    |
|               | Action (告警通知) 配置           |     |     |     |    |    |    |    |
|               | 故障情境模擬與完整驗證                |     |     |     |    |    |    |    |
| IV. 自動化響應與閉迴路 | 自動化修復腳本開發 (Remote Command) |     |     |     |    |    |    |    |
|               | 自動化閉迴路功能驗證                 |     |     |     |    |    |    |    |
|               | ITSM/維護流程整合規劃              |     |     |     |    |    |    |    |
| V. 報告撰寫       | 專題報告與成果整理                  |     |     |     |    |    |    |    |

## 需求分析

本專題的需求分析主要分為兩大類：功能性需求 (Functional Requirements) 和 非功能性需求 (Non-functional Requirements)。

### 功能性需求

| 類別   | 需求描述                                                                                        | 對應技術/實作點                  |
|------|---------------------------------------------------------------------------------------------|---------------------------|
| 環境部署 | 系統必須能夠在 PNETLAB 虛擬環境 中, 成功部署並運行 Docker 容器化的 Zabbix NMS 平台。                                  | Docker/PNETLAB 架構         |
|      | 系統必須能部署 SNMP 模擬器, 並使其在 PNETLAB 網路中扮演 Switch 設備的角色。                                          | SNMP 模擬器配置                |
| 數據採集 | 系統必須能夠透過 SNMP 協定 (Agentless 方式), 定期從模擬器中成功採集 Switch 的關鍵效能數據 (例如: 介面流量 OID, CPU/Memory OID)。 | SNMP v2c/v3, Zabbix Items |

|       |                                                                                                       |                                                          |
|-------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
|       | 系統必須能夠將原始數據(例如網路流量的 Byte 值)進行客製化計算(Custom Multiplier),轉換為易讀的單位(Mbps)。                                 | Zabbix Pre-processing<br>(預處理)                           |
| 監控與呈現 | 系統必須提供一個 Web 介面(Zabbix Frontend),能圖形化呈現監測數據的趨勢,並支援使用者自由選擇欲查看的設備或資訊。                                   | Zabbix<br>Graphs/Screens,React +<br>Chart.js,Dashboard設計 |
| 告警與通知 | 系統必須能設定 Triggers (觸發器),我們遵循業界 80% / 90% / 95% 原則設定閾值發出不同等級的告警,並結合 EWMA 統計模型,自動偵測突發事件與異常行為。            | Zabbix Triggers 設計,EWMA                                  |
|       | 系統必須能配置 Actions (動作),在觸發告警時,向指定用戶發送通知(Email 或其他通訊軟體),同時透過 WebSocket 即時推送至前端 Dashboard,確保使用者能快速掌握異常狀態。 | Zabbix Actions 配置,<br>WebSocket                          |
| 自動化響應 | 系統必須能針對高嚴重性告警,自動執行預先編寫的應急修復腳本。                                                                        | Zabbix Remote Command                                    |
|       | 系統必須能夠在文件上體現與 ITSM/工單系統的對接流程,完成故障處理的流程閉迴路。                                                            | ITSM/工單流程規劃                                              |
| 故障驗證  | 系統必須能透過修改模擬器數據的方式,驗證告警機制是否能被準確且即時地觸發。                                                                 | 實務故障情境模擬                                                 |

## 非功能性需求

| 類別 | 需求描述 | 衡量標準 |
|----|------|------|
|----|------|------|

|      |                                                                                                            |                                                                                        |
|------|------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| 效能   | 數據採集的延遲應盡可能低，確保數據的即時性。                                                                                     | 數據採集間隔與實際數據進入 Zabbix 的延遲盡量縮短。                                                          |
| 穩定性  | Zabbix NMS 平台應能 7x24 小時穩定運行，不會因數據量增長而崩潰。                                                                   | 系統需在模擬監測兩星期後仍能保持穩定。                                                                    |
| 可靠性  | 系統在執行自動化修復腳本時，必須具備日誌記錄功能，確保修復過程可被追蹤與審計。                                                                    | 自動化修復命令的執行狀態（成功/失敗）需被 Zabbix Log 完整記錄。                                                 |
| 可用性  | Web 介面應直觀易用，使監測狀態（Problem/OK）和告警訊息易於識別，並提供 Global Alerts 面板，讓使用者快速掌握設備狀態。                                  | 告警儀表板設計清晰，並使用不同顏色區分嚴重性。告警訊息可在短時間內被辨識。                                                  |
| 可維護性 | 系統應具備高度的可重現性與可移植性，方便未來教學與擴展。                                                                               | 專案所有配置（包括 PNETLAB 拓撲和 Docker Compose）需有詳細文件。                                           |
| 安全性  | 前端透過 Node.js 中介層取得監控資料，不直接連線 Switch 或 SNMP 裝置，API 層負責授權驗證，確保系統安全性與資料一致性。自動化修復採用的 Remote Command 需符合最小權限原則。 | 所有資料請求必須經過授權驗證，確保使用者操作安全且資料流向可追蹤。Remote Command 執行用戶僅具備執行指定腳本的權限，不具備 Root 權限或任意文件存取權限。 |

## 組員工作分配

組長：林采儀

1. 平台核心配置：負責 Docker 環境配置、Zabbix Server 的部署與核心參數設定。
2. 數據專業：專責 SNMP OID 採集、**Custom Multiplier** 預處理實作與驗證（將 Byte 轉換為 Mbps）。
3. 告警與自動化：負責 Triggers 邏輯設定、**Remote Command** 應急修復腳本開發，以及自動化閉迴路功能的邏輯驗證。
4. 報告撰寫：擔任報告技術內容的總撰寫與校對。

· 組員：許舜曄

1. 虛擬化基石與數據餵入：PNETLAB 環境配置、SNMP 模擬器部署與 OID 數據初始餵入。
2. 網路連通性與腳本開發：負責虛擬網路 IP 規劃，並協作 **Remote Command** 應急修復腳本開發。
3. 系統穩定與閉迴路環境：專責環境 **7x24** 小時穩定運行，保障自動化閉迴路功能成功執行。

· 組員：伍翊瑄

1. 數據驗證：協助進行數據的趨勢分析、圖表截取，並對報告數據進行呈現上的優化。
2. 儀表板設計：專責 Zabbix Web Frontend 的儀表板 (Dashboard) 設計與實作，確保數據可視化效果專業且具備實務可讀性。
3. 流程優化：負責告警信息的格式化，使最終告警能清晰地與 ITSM 流程對接。

## 系統特色

---

### 1. 高擬真的虛擬化實戰環境 (PNETLAB Integration)

- 特色描述：系統將 NMS 平台與監測對象 (SNMP 模擬器) 完整部署於 **PNETLAB** 虛擬化平台 中，取代昂貴的實體 Switch 設備。
- 價值：提供了低成本、高效率且可重現的網路監測實驗環境，極大地提高了專題的實用性與教學價值。

## 2. 高效的容器化部署架構 (Docker/Zabbix)

- 特色描述：採用 **Docker** 容器化技術 部署 Zabbix Server、Web 介面與資料庫。
- 價值：實現了系統的快速部署與高可移植性。只需單一指令即可啟動或遷移整個 NMS 環境，大幅降低了系統維護與環境架設的複雜度。

## 3. 精準的數據客製化處理 (Custom Multiplier)

- 特色描述：系統不僅能透過 SNMP OID 採集原始數據，更能應用 Zabbix 的 **Custom Multiplier** 預處理機制，將網路流量的原始位元組 (Byte) 數值準確轉換為實務上常用的 **Mbps** 單位。
- 價值：解決了原始數據的判讀困難，確保監測圖表的專業性與實務參考價值。

## 4. 即時監控與視覺化呈現 (Real-Time Dashboard)

- 特色描述：前端提供直觀且互動性高的 Dashboard，整合多種監測指標，並透過功能面板設計，讓使用者快速掌握設備健康狀態與異常事件。
- 價值：強化使用者操作體驗與資訊可視化，將複雜監控數據轉化為直觀圖表與即時警示，縮短異常反應時間。

## 5. 完整的故障情境模擬與閉迴路驗證 (Full-Cycle Simulation)

- 特色描述：本專案的核心特色是透過 手動修改 SNMP 模擬器的數據，主動創造網路故障 (如流量超載) 情境，並系統化驗證 Zabbix 的 Triggers 是否能即時觸發告警，並在數據恢復後正確解除告警。
- 價值：成功演示了 NMS 從「異常發生」到「告警通知」再到「問題解決」的完整生命週期，確保系統的 預警機制可靠性，解決了在實體環境中難以穩定重現故障情境的問題。

## 6. 強化告警事件管理與自動化閉迴路 (Advanced Event Correlation and Self-Healing)

- 特色描述：系統規劃並實作輕量級的自動化響應機制，透過 Zabbix 的 Actions 機制，在特定高嚴重性告警觸發時，自動執行 **Remote Command** 嘗試執行應急修復腳本。此外，本專題在文件層面整合了 ITSM/工單流程 概念，將告警通知作為連線到企業維護流程的起點。

- 價值：將監控能力從單純的「通知」升級到「事件處理」。這不僅證明系統具備自我修復 (**Self-Healing**) 的潛力，同時展現了與企業維護流程(工單系統)的整合思維，極大提升了系統的專業度與實務應用價值。

## 7. 模組化與可擴展性設計

- 特色描述：所有監測項目皆透過 **Zabbix Template** 集中管理，且監測方式採用 **Agentless (SNMP)** 設計。
- 價值：系統具備高度的模組化，未來可以輕鬆擴展到監測更多虛擬或實體 Switch 設備，只需將新設備連結至現有樣板，即可快速納入監測範圍。

# 可解決哪些問題

---

本專題實作的 Zabbix NMS 模擬監測系統，主要針對網路監控領域和專題實作的挑戰，解決了以下五個關鍵問題：

## 1. 解決實體設備資源匱乏與成本高昂的問題

- 問題：實體網路硬體價格昂貴，難以取得多樣型號或大規模用於教學與測試，且設備損耗成本高。
- 解決：透過 **PNETLAB** 虛擬化平台與 **SNMP** 模擬器 的結合，成功在虛擬環境中重現真實的監測場景。這使得網路監測系統的開發、測試與教學得以在低成本、零損耗的環境下進行。

## 2. 解決網路故障反應遲緩的問題

- 問題：傳統依賴人工巡檢或被動等待使用者回報故障的方式，反應時間長，無法在問題發生初期即時介入。
- 解決：實現了 **Zabbix** 的主動式監測。透過精確設定 **Triggers**，系統能在 Switch 數據(如流量、CPU)達到預設門檻時立即觸發告警，將反應時間從數小時縮短至數分鐘甚至數秒。

## 3. 解決數據單位混亂與判讀困難的問題

- 問題：透過 SNMP 收集到的原始網路流量數據通常是以 **Byte/s** 為單位，對於網路工程師或管理人員來說，難以直觀判讀與評估網路頻寬的使用狀況。
- 解決：利用 Zabbix 的 **Custom Multiplier** 預處理功能，將原始數據精準地轉換為行業標準的 **Mbps** (每秒百萬位元) 單位，極大地提高了監測圖表的可讀性與實務參考價值。

#### 4. 解決 NMS 系統部署與遷移複雜的問題

- 問題：傳統部署 NMS 需要花費大量時間設定作業系統、資料庫和 Zabbix 服務，環境依賴性高，難以在不同環境間遷移。
- 解決：採用 **Docker** 容器化 部署 Zabbix 平台。這使得整個 NMS 環境成為一個高度可移植的獨立包，解決了環境配置依賴問題，系統可以在任何支援 Docker 的環境中快速重建與啟動。

#### 5. 解決 NMS 告警流程無法完整驗證的問題

- 問題：在沒有真實故障或無法控制的實體環境中，很難穩定地、重複地測試「故障發生→發出告警→故障解除→恢復通知」的完整告警生命週期。
- 解決：專題透過修改 **SNMP** 模擬器數據，實現了對網路故障情境的精準控制與重現。這使得系統能夠系統化地驗證所有 Triggers 與 Actions 的配置是否有效，確保告警機制的可靠性。

#### 6. 解決告警洪水、告警疲勞與維護流程脫節的問題

- 問題：傳統 NMS 系統常因閾值設定過多，導致大量孤立警報 (Alert Flood)，造成維護人員的「告警疲勞 (**Alert Fatigue**)」。更重要的痛點是，警報發出後，與後續的維護、開立工單流程往往是脫節的，缺乏自動化的初始響應機制。
- 解決：
  1. 自動化應急響應：系統實作了 **Zabbix Remote Command** 自動化修復功能。在收到高嚴重性告警時，系統會自動執行初階修復腳本，嘗試在人工介入前解決或緩解問題，將被動等待轉變為主動預防。
  2. 流程閉迴路整合：在文件上體現了與 **ITSM/Service Desk** 工單系統的對接思維。將最終的告警通知視為連線到企業標準維護流程(開立工單)的起點，從而確保告警事件能無縫銜接到企業的標準維護與追蹤流程，完成從 偵測 到 響應 到 維護 的全程閉迴路。

## 系統使用對象

---

| 對象     | 角色與系統互動方式                                                                        | 系統提供的主要價值                                                           |
|--------|----------------------------------------------------------------------------------|---------------------------------------------------------------------|
| 網路工程師  | 負責日常的網路設備運行狀態監控、故障初步診斷與流量趨勢分析。                                                   | 提供即時、準確的 Switch 效能指標，並作為自動化響應成功或失敗後的後續人工介入依據。                       |
| 系統管理員  | 負責 Zabbix 平台的維護、Template/Triggers 的配置與優化，以及 <b>Remote Command</b> 腳本 和告警通知機制的管理。 | 透過 Docker/Zabbix 架構實現高效的平台部署與管理，並可彈性調整監測規則，配置系統的自我修復邏輯。             |
| 網路維護人員 | 專門負責接收並處理來自 NMS 的告警通知，執行標準化故障排除流程，並追蹤自動化修復的結果。                                   | 提供清晰的 告警通知 與 自動化響應狀態，幫助快速識別問題，並將故障事件無縫銜接到 <b>ITSM/工單系統</b> ，提升維護效率。 |

## 系統環境

本專題的系統環境主要依賴 **PNETLAB** 虛擬化平台 建立一個隔離、可控的網路環境，並採用容器化技術部署核心服務。

### 硬體與虛擬化平台 (Hardware and Virtualization)

| 類別           | 規格描述              | 備註                            |
|--------------|-------------------|-------------------------------|
| 虛擬化平台        | PNETLAB           | 作為承載所有虛擬設備的基礎環境。              |
| PNETLAB 主機資源 | CPU: 建議 4 核心以上    | 確保 Zabbix Server 和多個模擬器能穩定運行。 |
|              | RAM: 建議 8 GB 以上   |                               |
| 內部虛擬機器       | 儲存空間: 建議 16 GB 以上 | 滿足 Docker 容器和系統運行的基本需求。       |



|        |                 |                                                                       |
|--------|-----------------|-----------------------------------------------------------------------|
| 虛擬網路拓撲 | 採用 PNETLAB 內部網路 | 建立一個虛擬 LAN (Local Area Network), 使 NMS Server 與 SNMP 模擬器處於同一網段, 方便通訊。 |
|--------|-----------------|-----------------------------------------------------------------------|

## 軟體與 NMS 平台 (Software and NMS Stack)

| 類別     | 元件名稱                                | 建議版本                         | 功能與用途                                                                           |
|--------|-------------------------------------|------------------------------|---------------------------------------------------------------------------------|
| 作業系統   | Ubuntu Server 或 CentOS              | LTS 版本 (例如 Ubuntu 22.04 LTS) | PNETLAB 網路環境中, 設備的基礎作業系統, 用於運行 Docker 服務。                                       |
| 容器化技術  | Docker Engine & Docker Compose      | 最新穩定版                        | 用於快速部署前後端, Zabbix Server, Database 等多個服務, 降低環境依賴與維護成本, 支援 Traefik / Nginx 反向代理。 |
| NMS 平台 | Zabbix Server / Zabbix Web Frontend | 6.0 LTS 或 7.0 (最新版)          | 核心網路監測系統, 負責數據收集、圖表繪製與告警處理。                                                     |
| 資料庫    | PostgreSQL 或 MySQL                  | 適用於 Zabbix 的版本               | 儲存所有監測數據、配置資訊與歷史趨勢。                                                             |
| 資料仲介   | Node.js API Server                  | 最新穩定版                        | 作為前端與 Zabbix/資料庫的資料仲介, 負責 API 整合、資料轉換、授權控管及資料流向管理, 提供統一資料介面與快取能力。               |

## 專題核心技術元件 (Core Implementation Components)

| 類別   | 元件名稱                                      | 用途與實作點                                                  |
|------|-------------------------------------------|---------------------------------------------------------|
| 設備模擬 | SNMP 模擬器                                  | 取代實體 Switch, 用於模擬 Switch 設備對 SNMP 查詢的回覆數據。              |
| 通訊協定 | SNMP (Simple Network Management Protocol) | Agentless 監測 的核心協定, 用於 Zabbix 向設備查詢 OID 數據。             |
| 數據轉換 | Zabbix Pre-processing (預處理)               | 實作 Custom Multiplier, 將原始 Byte 數據轉換為 Mbps。              |
| 告警響應 | Zabbix Actions & Remote Command           | 實作自動化響應機制。Actions 負責觸發流程, Remote Command 負責執行輕量級應急修復腳本。 |
| 流程規劃 | ITSM/工單系統對接流程                             | 文件上規劃與說明告警事件如何對接至後續的企業維護流程, 完成閉迴路追蹤。                    |
| 故障演示 | 模擬器數據修改機制                                 | 專門用於手動修改模擬器餵入的 OID 數值, 以觸發告警並驗證自動化修復功能。                 |

## 應用情境

### 網路監測與故障模擬實戰情境

| 情境名稱            | 實務痛點                                      | 系統帶來的改變                                                                      |
|-----------------|-------------------------------------------|------------------------------------------------------------------------------|
| 情境一: 流量擁塞的預警與管理 | 客戶抱怨網路慢了才去查, 發現某個端口早已跑滿, 業務中斷已成定局, 但為時已晚。 | 系統為流量數據設定 Triggers。在端口達到 <b>80%</b> 負載時, 即刻發出預警通知。讓工程師有時間在服務中斷前介入處理, 實現主動預警。 |
| 情境二: 設備「過勞死」的   | Switch 設備因 CPU 或記憶體耗盡而突然當機, 維護人員只能在       | 系統持續監測設備的「健康指標」。一旦發現 CPU 或記憶體使用率異常飆升, 立即發出高嚴重性告                              |

|                    |                                        |                                                                                                            |
|--------------------|----------------------------------------|------------------------------------------------------------------------------------------------------------|
| 預防與自動化應急           | 接到大量投訴後緊急重啟，耗費大量人力。                    | 警，並自動觸發 <b>Remote Command</b> ，嘗試執行初階的應急修復腳本，避免設備當機。                                                       |
| 情境三：複雜環境的快速部署與高移植性 | 架設一套新的監控系統耗時費力，配置繁瑣，環境依賴性高，難以在不同環境間遷移。 | 運用 <b>Docker</b> 容器化技術，將整個 NMS 系統打包。只需數分鐘即可在 PNETLAB 或任何雲端環境中一鍵重啟整套監控系統。這使得團隊能夠將精力集中於監測邏輯與自動化腳本的優化，而非環境建置。 |
| 情境四：故障修復後的流程追蹤與閉迴路 | 故障排除後，缺乏正式記錄和工單追蹤，無法對故障原因進行根本性分析和審計。   | 系統在成功或失敗執行自動化修復後，將最終的告警通知視為與 <b>ITSM</b> 系統對接的憑證。這確保了從**「偵測」到「響應」再到「維護追蹤」**的完整閉迴路管理。                       |

## 結語

本專題成功實作了一套基於 **Docker/Zabbix** 架構的 NMS 模擬監測系統。我們透過 **PNETLAB** 虛擬化平台 和 **SNMP** 模擬器 解決了實體設備資源受限的挑戰，並藉由 **Custom Multiplier** 確保了數據的專業性和準確性。

核心價值在於我們不僅完成了從「數據採集」到「主動預警」的基礎目標，更進一步在架構中實現了「自動化響應與流程閉迴路」的高階功能。透過配置 **Zabbix Remote Command**，本系統演示了從故障發生到系統嘗試自我修復的完整生命週期。

我們的成果證明了在低成本、高移植性的環境下，能夠建立一套符合下一代 **NMS** 趨勢的解決方案。本專題為網路監測領域從被動式維護轉向主動式預測與自動化應急提供了強大的實戰驗證平台。